C++中 vector 动态数组的使用详解

`vector` 是 C++ 标准库中最常用的**动态数组**容器，你可以把它理解成「长度能自动变化的数组」——解决了普通数组长度固定、无法灵活扩容的痛点。

一、先搞懂核心作用（新手视角）

普通数组（比如 `int arr[5];`）的问题：

- 长度在定义时就固定死了，想加第6个元素根本做不到；
- 想知道数组里实际存了多少元素，还得自己记，很麻烦。

而 `vector` 帮你解决了这些问题：

1. **自动扩容**：存满了会自动申请更大的内存，把数据搬过去，不用你手动处理；
2. **灵活操作**：可以轻松添加、删除元素，随时获取当前元素个数；
3. **用法和数组相似**：支持 `[]` 下标访问（比如 `vec[0]`），上手成本低。

二、基础用法（结合你的代码示例）

先看最核心的使用步骤，再解释你代码里的 `vector`：

1. 基础使用模板

Code block

```
#include <iostream>
// 必须包含这个头文件才能用 vector
#include <vector>
using namespace std;

int main() {
    // 1. 定义一个存储整数的 vector（空数组）
    vector<int> nums;

    // 2. 往里面加元素（自动扩容）
    nums.push_back(2);  // 现在 nums = [2]
    nums.push_back(7);  // 现在 nums = [2,7]
    nums.push_back(11); // 现在 nums = [2,7,11]

    // 3. 用下标访问（和普通数组一样）
    cout << nums[0] << endl;  // 输出 2
    cout << nums[1] << endl;  // 输出 7

```

```
    // 4. 获取元素个数（对应你代码里的 nums.size()）
    cout << nums.size() << endl;  // 输出 3

    // 5. 返回 vector（对应你代码里的 return {i,j}）
    vector<int> res = {0, 1};  // res = [0,1]
    return 0;
}
```

2. 你代码里的 `vector` 解释

Code block

```
// 函数返回值：返回一个存储整数的 vector（存两个索引）
vector<int> twoSum(vector<int>& nums, int target) {
    // ...
    return {i, j};  // 返回包含 i 和 j 的 vector
    // ...
    return {};      // 返回空的 vector（兜底）
}
```

- `vector<int>& nums`：`&` 表示「引用」，目的是避免拷贝整个数组（节省内存、提高速度），你可以先理解成「直接用原数组，不是复制一份」；
- `return {i,j}`：C++11 及以后支持的简化写法，等价于 `vector<int> result = {i,j}; return result;`，本质是返回一个包含两个整数的动态数组。

三、vector 常用场景（新手必知）

1. **刷题/算法场景**：像你做的「两数之和」，需要返回不确定长度的结果（比如两个索引），用 `vector` 最方便；
2. **日常开发场景**：需要动态添加数据（比如读取文件内容、存储用户输入），普通数组不够灵活，就用 `vector`；
3. **替代普通数组**：新手可以优先用 `vector`，避免普通数组的越界、长度固定等问题。

总结

1. `vector` 是 C++ 的**动态数组**，长度可自动变化，比普通数组更灵活；
2. 核心用法：`push_back()` 加元素、`size()` 查长度、`[]` 访问元素；
3. 你代码里的 `vector` 主要用来「接收输入数组」和「返回结果（两个索引）」。

如果想进一步了解 `vector` 的其他用法（比如删除元素、初始化、遍历），可以告诉我，我再补充~

（注：文档部分内容可能由 AI 生成）